

A Project Report
submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

on

iTantra

*Indian Multilingual TTS & STT Aided Neural Transceiver —
Radio Access for Low Bitrate Links*

Smart India Hackathon 2026 · Problem Statement **SIH26173**
Indian Space Research Organisation, Department of Space

Submitted by

Under the guidance of

Assistant Professor
Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VIGNANA BHARATHI INSTITUTE OF TECHNOLOGY

(A UGC Autonomous Institution, Approved by AICTE,

Accredited by NBA & NAAC A Grade, Affiliated to JNTUH)

Aushapur (V), Ghatkesar (M), Medchal–Malkajgiri (D), Hyderabad – 501 301

2026 – 2027

CERTIFICATE

This is to certify that the project report entitled “**iTantra — Indian Multilingual TTS & STT Aided Neural Transceiver: Radio Access for Low Bitrate Links**” is a bonafide record of the work carried out by

in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from Vignana Bharathi Institute of Technology, Hyderabad, an autonomous institution affiliated to Jawaharlal Nehru Technological University Hyderabad, during the academic year 2026–2027.

The work presented in this report was carried out under our supervision and guidance. The results embodied in this report have not been submitted to any other university or institution for the award of any degree or diploma.

The project addresses Problem Statement **SIH26173**, issued by the Indian Space Research Organisation, Department of Space, for the Smart India Hackathon 2026.

Internal Guide

Assistant Professor
Department of CSE

Head of the Department
Department of CSE

Internal Examiner

External Examiner

DECLARATION

We hereby declare that the project report entitled “**iTantra — Indian Multilingual TTS & STT Aided Neural Transceiver: Radio Access for Low Bitrate Links**”, submitted to the Department of Computer Science and Engineering, Vignana Bharathi Institute of Technology, Hyderabad, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology, is a record of original work carried out by us.

The design, the engineering specification and the evaluation methodology presented in this report are our own. Every third-party component used in the system is open source, and each is listed with its licence in Chapter 4 and Chapter 9. Two components carrying restrictive licences — espeak-ng under GPL-3.0 and Meta MMS under CC-BY-NC — are disclosed explicitly rather than left to be discovered.

We further declare that the performance figures reported in Chapter 8 are stated as **design targets** and are clearly identified as such, since implementation is in progress at the time of submission. No figure in this report is presented as a measured result unless it has been produced by the evaluation harness described in Chapter 7.

The work contained in this report has not been submitted to any other university or institution for the award of any degree or diploma.

Place: Hyderabad

Date: _____

Signature of the candidates

ACKNOWLEDGEMENT

We express our sincere gratitude to our internal guide, Department of Computer Science and Engineering, for the continuous guidance, technical direction and encouragement offered throughout the course of this project. The clarity of the engineering specification presented in this report owes a great deal to that supervision.

We are thankful to the Head of the Department of Computer Science and Engineering for providing the facilities and the academic environment necessary to carry out this work, and for the encouragement to participate in the Smart India Hackathon.

We extend our thanks to the Principal and the Management of Vignana Bharathi Institute of Technology for their constant support and for the infrastructure made available to us.

We gratefully acknowledge the Indian Space Research Organisation, Department of Space, for framing Problem Statement SIH26173. The problem is unusually well posed: it states its evaluation criteria and their weights explicitly, and that clarity shaped the evaluation methodology described in Chapter 7 of this report.

We also acknowledge the open-source community whose work this system is built upon — in particular AI4Bharat and IIT Madras for Indian-language speech models and corpora, the k2-fsa project for sherpa-onnx, and the Rhasspy project for the Piper synthesis voices. Without these freely available resources, an entirely offline multilingual speech system would not be achievable within the scope of an undergraduate project.

Finally, we thank our families and our fellow students for their support and encouragement throughout the duration of this work.

Project team

ABSTRACT

In alert and distress scenarios, transmitting voice is more inclusive than transmitting written text, because it does not require the sender or the receiver to be literate. However, voice is data-intensive. Reproducing intelligible speech requires approximately 256 kbps of raw pulse-code modulated audio, and even the most efficient open-source speech codec available, Opus, has a practical intelligibility floor of about 6 kbps. The narrowband radio links that remain usable when terrestrial infrastructure has failed — LoRa, HF data links and narrowband satellite bursts — provide only a few hundred bits per second. Voice communication over such links is therefore not merely difficult; it is bounded away from possibility, because audio codecs compress the waveform and an intelligible waveform has an irreducible size.

This project presents **iTantra**, an Android application that resolves this contradiction by removing the waveform from the link entirely. Speech is recognised on the transmitting handset, the resulting text is transmitted as a compact binary frame of a few dozen bytes, and speech is re-synthesised on the receiving handset. Both operators only ever speak and listen; neither reads nor types at any point. The system supports ten Indian languages — Hindi, Gujarati, Marathi, Kannada, Malayalam, Tamil, Telugu, Odia, Bengali and English — and executes entirely on the device, with no cloud service, no SIM, no proprietary software development kit and no network permission declared in the application manifest.

Compression is achieved in three graceful stages. Semantic transcoding converts speech to text, giving a reduction of approximately 770 to 1. Script packing exploits the fact that the frame header already declares the language, allowing each Indic character to be represented in a single byte instead of the three that UTF-8 requires. Template coding maps common operational sentences to single-byte identifiers, which additionally delivers cross-language operation without any translation model, since each device holds the template table in all ten languages. A three-second Hindi sentence occupying 96 000 bytes as raw audio is transmitted in 44 bytes, a reduction of 2 182 to 1.

The application is instrumented from the outset. Word error rate is evaluated per language at four signal-to-noise ratios against held-out public benchmark corpora, and a separate critical-term error rate is reported with and without contextual biasing. Latency is measured per pipeline stage inside the shipped application rather than externally, and efficiency is reported as model size, memory footprint and idle CPU usage on an entry-tier handset after a thirty-minute thermal soak.

Keywords: speech recognition, speech synthesis, low-bitrate communication, semantic compression, Indian languages, on-device inference, LoRa, disaster communication.

TABLE OF CONTENTS

	Certificate	ii
	Declaration	iii
	Acknowledgement	iv
	Abstract	v
	List of Figures and Tables	viii
1	INTRODUCTION	1
1.1	Overview	1
1.2	Problem Statement	1
1.3	Objectives	3
1.4	Scope and Limitations	3
2	LITERATURE SURVEY	5
2.1	Speech Recognition for Indian Languages	5
2.2	On-Device Speech Synthesis	5
2.3	Messaging over Low-Bitrate Radio	7
2.4	Summary of the Gap	7
3	SYSTEM ANALYSIS	8
3.1	Existing System	8
3.2	Proposed System	8
3.3	Advantages of the Proposed System	10
3.4	Feasibility Study	10
4	SYSTEM REQUIREMENTS	12
4.1	Hardware Requirements	12
4.2	Software Requirements	12
4.3	Functional Requirements	14
4.4	Non-Functional Requirements	14
5	SYSTEM DESIGN	16
5.1	System Architecture	16
5.2	Module Description	17
5.3	Wire Protocol Design	17

6	IMPLEMENTATION	19
6.1	Speech Recognition Module	19
6.2	Speech Synthesis Module	20
6.3	Compression Module	20
6.4	Transport Module	22
7	TESTING AND EVALUATION	24
7.1	Evaluation Methodology	24
7.2	Test Cases	25
8	RESULTS AND DISCUSSION	26
8.1	Compression Results	26
8.2	Accuracy, Latency and Efficiency Targets	27
8.3	Discussion	28
9	CONCLUSION AND FUTURE SCOPE	29
	REFERENCES	-1

LIST OF FIGURES AND TABLES

Figures

1.1	Comparison of data rates on a logarithmic scale	1
5.1	System architecture and end-to-end signal path	16
5.2	Frame format of the iTantra wire protocol	17
6.1	Demonstration and deployment topologies	22

Tables

3.1	Size of a three-second sentence in each representation	9
4.1	Hardware requirements	12
4.2	Software requirements and licences	13
5.1	Module description	17
5.2	Frame field description	18
6.1	Effect of quantisation on model size and speed	19
6.2	Comparison of the four transport implementations	22
7.1	Test cases and expected results	25
8.1	Frame size and time on a 300 bps link	26
8.2	Word error rate targets at four noise levels	27
8.3	Latency and efficiency targets	28

CHAPTER 1

INTRODUCTION

1.1 Overview

Communication is the first casualty of a disaster. When mobile towers fail, the equipment that continues to work is the handheld radio set, because it requires no infrastructure at all — only a battery and an antenna. Such radios, however, provide a narrowband data channel of a few hundred bits per second, determined by terrain, power budget and spectrum allocation. A voice call cannot pass through such a channel.

This is not a limitation that a better codec will eventually remove. Reproducing a human voice with sufficient fidelity for words to remain distinguishable requires sampling air pressure sixteen thousand times per second at sixteen bits, producing approximately 256 000 bits for every second of speech. Opus, the most capable open-source speech codec available, reduces this to about 6 000 bits per second at the cost of a thin, robotic timbre; below that rate intelligibility collapses. The gap between that floor and the capacity of a narrowband link remains a factor of twenty, and it is a bound rather than an engineering deficiency, because audio codecs compress the shape of the pressure wave and an intelligible waveform has an irreducible size.

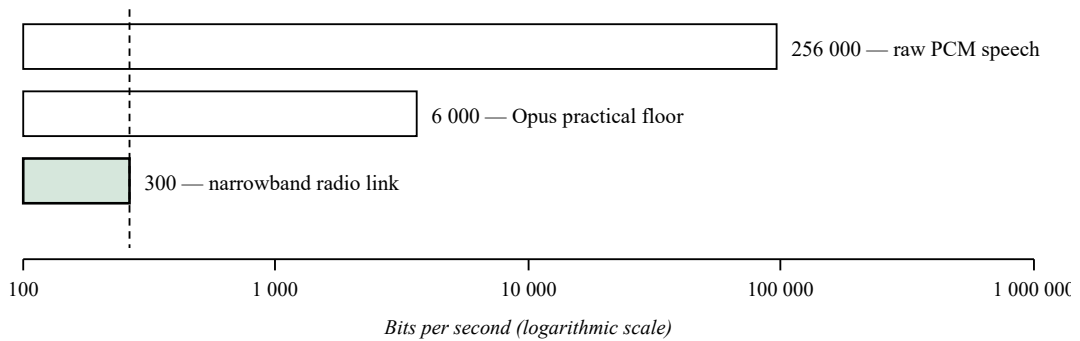


Figure 1.1: Comparison of data rates on a logarithmic scale. Both audio representations lie far to the right of the capacity of the link.

1.2 Problem Statement

Problem Statement SIH26173, issued by the Indian Space Research Organisation, requires the development of an Android application containing lightweight and accurate speech-to-text and text-to-speech models for ten Indian languages, executing locally on a low-power device. The speech-to-text module must activate after detecting pauses and stoppages, form complete sentences from the speech detected, and stream the resulting

data efficiently through a Wi-Fi or Bluetooth connected embedded device or another handset running the same application. The text-to-speech module must convert received text into intelligible speech played as a voice note, and alert-type messages must be announced at the highest volume in a non-interruptible manner. Two handsets running the same application, one in speech-to-text mode and one in text-to-speech mode, must together behave as a walkie-talkie operated by a push-to-talk control, and as a telephone when that control is disabled.

1.3 Objectives

The objectives of this project are as follows.

1. To design and implement an Android application that converts speech to text on the transmitting device and text back to speech on the receiving device, so that both users interact with the system only by speaking and listening.
2. To support ten Indian languages with quantised on-device acoustic and synthesis models, each language pack occupying less than seventy megabytes.
3. To define a compact binary wire protocol that transmits a complete spoken sentence in fewer than sixty bytes, so that transmission is feasible over links providing only a few hundred bits per second.
4. To implement voice activity detection and endpointing so that recognition is triggered by natural pauses, and idle processor usage remains below two per cent.
5. To provide a transport abstraction with multiple implementations, including one that is serial in shape so that the same frames may be carried by a long-range radio module.
6. To deliver alert messages at maximum volume in a non-interruptible manner, including on a handset that is locked and silenced.
7. To instrument the application so that accuracy, latency and efficiency are measured and exported rather than asserted.

1.4 Scope and Limitations

The scope of the project is a symmetric Android application in which every device runs both the recognition and the synthesis halves, and the operating role is a runtime mode rather than a build variant. The system operates entirely offline at run time. Language packs may be fetched once during initial setup, after which no network access occurs.

The following limitations are stated explicitly rather than concealed. The identity of the speaker is not preserved, since the waveform is discarded; this is compensated by a sender identifier carried in the frame header and a distinct synthesis voice for each node. Prosody and emotional content are likewise not preserved, and are compensated by an explicit priority class for alert traffic. Background audio is discarded entirely, which is accepted as a consequence of the bandwidth budget. Cross-language delivery is available only for template-coded messages, because it is a property of a shared table rather than of a translation model; free-form messages are delivered in the language in which they were

spoken. Finally, recognition of code-mixed Hindi and English speech is a known weakness, which is disclosed together with the mitigation adopted.

CHAPTER 2

LITERATURE SURVEY

2.1 Speech Recognition for Indian Languages

Automatic speech recognition has progressed from hidden Markov model systems to end-to-end neural architectures. The Conformer architecture proposed by Gulati and others combines convolutional blocks, which capture local acoustic events such as plosive bursts and formant transitions, with self-attention blocks that supply sentence-level context. This combination has become the standard encoder for streaming recognition because it achieves competitive accuracy at a computational cost acceptable on mobile hardware.

For Indian languages, the AI4Bharat initiative at IIT Madras has released the IndicConformer family of acoustic models, trained specifically on Indian language speech, together with the IndicSUPERB, Kathbath and IndicVoices evaluation corpora. These resources are significant for the present work because general multilingual models trained predominantly on European languages perform poorly on Indic phonology and on scripts such as Odia and Kannada. The Vosk toolkit provides smaller streaming models with immediate Android support and lower accuracy, and is therefore useful as a development baseline. OpenAI Whisper offers strong accuracy but is not a streaming architecture and is computationally heavy for entry-tier mobile processors.

2.2 On-Device Speech Synthesis

Earlier neural synthesis pipelines required two models, one converting text to a spectrogram and a second converting the spectrogram to a waveform, which proved too slow for mobile devices. The VITS architecture described by Kim, Kong and Son performs both operations in a single pass and incorporates a duration predictor, which is the component responsible for natural rhythm and which prevents synthesised speech from sounding metronomic. The Piper project provides VITS voices already exported to the ONNX interchange format, and AI4Bharat provides Indic-TTS voices of higher naturalness for Indian languages.

An observation relevant to this project is that two of the four stages of a synthesis pipeline are conventional software engineering rather than machine learning. Text normalisation, which determines whether a numeral is read as a quantity or dialled digit-wise, and phonemisation, which converts orthography to phoneme sequences, together account for a substantial part of perceived output quality. Indic scripts being largely phonetic makes

phonemisation considerably more reliable for the target languages than it would be for English.

2.3 Messaging over Low-Bitrate Radio

Several open-source systems transmit text over long-range, low-bitrate radio links. The Meshtastic project implements text messaging over LoRa with an Android client connected by Bluetooth, and its firmware informs the design of framing, addressing, relay behaviour and duplicate suppression. The Briar project demonstrates robust Bluetooth and Wi-Fi transport on Android, including discovery, reconnection and the handling of degraded states. APRSdroid connects Android devices to amateur radio equipment over a Bluetooth serial profile, and is a working demonstration of the deployment path adopted in this project.

The common characteristic of these systems is that the user types the message and reads the reply. They therefore solve the transport problem while leaving the accessibility problem untouched, which is precisely the gap that Problem Statement SIH26173 identifies.

2.4 Summary of the Gap

The literature therefore contains mature solutions to each half of the problem, and no solution that joins them. On-device speech recognition and synthesis for Indian languages are available under permissive licences and are computationally feasible on mobile hardware. Long-range text messaging over narrowband radio is likewise a solved problem with several working open-source implementations. What does not exist is a system that places speech interfaces at both ends of such a link so that the operators never read or type, which is what makes the system usable by a person who cannot read.

The contribution of this project is therefore not a new speech model or a new radio protocol, but the composition of the two, together with a compression scheme that exploits a property unavailable to general-purpose systems: because both endpoints are controlled and the frame header declares the language, Indic text may be packed into a single-byte alphabet, and common operational sentences may be reduced to single-byte identifiers.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

Existing approaches to voice communication over constrained links fall into two categories, neither of which is satisfactory for the scenario described in the problem statement. The first category consists of audio codecs, of which Opus is representative. These systems preserve the voice of the speaker and require no recognition model, but they cannot operate below approximately six thousand bits per second without losing intelligibility, and are therefore unusable on the links that remain available during a disaster.

The second category consists of text messaging systems over long-range radio, of which Meshtastic is representative. These systems operate comfortably within the available bandwidth, but they require the sender to type the message and the receiver to read it. Both operations exclude a non-literate user, are impractical for an operator wearing gloves or working in darkness, and are slow under stress: speaking a sentence takes approximately three seconds, whereas typing the same sentence in Devanagari on a mobile keyboard takes approximately forty.

3.2 Proposed System

The proposed system removes the waveform from the link. Speech is recognised on the transmitting handset and converted to text; the text is compressed and transmitted as a compact binary frame; and the text is re-synthesised as speech on the receiving handset. The interface presented to both operators is voice, and the representation carried by the link is text. Table 3.1 gives the size of a representative three-second Hindi sentence in each representation.

Representation	Size	Time on a 300 bps link	Remark
Raw PCM, 16 kHz, 16-bit mono	96 000 B	43 min	Output of the microphone
Opus at 16 kbps	6 000 B	2.7 min	Good quality voice call
Opus at 6 kbps	2 250 B	60 s	Practical intelligibility floor
UTF-8 text	99 B	2.6 s	Output of the recogniser
Packed text with frame header	44 B	1.2 s	Single-byte Indic alphabet
Template code with frame header	13 B	0.3 s	One byte of payload

Table 3.1: *Size of a three-second Hindi sentence in each representation, and the time required to transmit it over a 300 bps link.*

3.3 Advantages of the Proposed System

1. **Inclusive by construction.** Neither operator reads or types at any point, so the system is usable by a person who cannot read, by an operator wearing gloves, and by an operator who cannot look at the screen.
2. **Feasible on links that carry no voice.** A complete sentence occupies 44 bytes, which is transmitted in 1.2 seconds over a 300 bps link on which a voice call is impossible.
3. **Entirely offline.** All inference is performed on the device. The application declares no internet permission, so the property is enforced by the operating system rather than merely asserted by the developers.
4. **Cross-language operation without a translation model.** Because every device holds the template table in all ten languages, a template-coded alert is announced in whichever language the receiver has selected.
5. **Deployable on existing hardware.** The application runs on low and mid-range handsets. Long-range operation requires approximately ₹1 500 of radio hardware per pair and no change to the handset software.

3.4 Feasibility Study

3.4.1 Technical feasibility

Every component required by the system exists today, is released under an open-source licence, and executes on ARM processors. Quantised acoustic models occupy approximately thirty-five megabytes per language and synthesis voices approximately twenty-six megabytes, which is within the storage budget of an entry-tier handset. Reference implementations of the transport layer exist in the projects surveyed in Chapter 2. No component of the system requires research; the work is one of composition and engineering.

3.4.2 Economic feasibility

The system incurs no running cost. There is no server to operate, no data plan to purchase and no per-message charge. It executes on handsets that users already possess. Long-range deployment requires two radio nodes costing approximately ₹1 500 in total, each consisting of an ESP32 microcontroller and an SX1276 transceiver operating in the licence-free 865–867 MHz band allocated in India.

3.4.3 Operational feasibility

The operating model is that of a walkie-talkie, which is already familiar to the intended users. The transmit control occupies at least one third of the screen and is additionally bound to the volume-down hardware key, so that the system may be operated with the screen switched off. Every state change is confirmed by haptic feedback and by a spoken cue rather than by a text dialogue alone.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Component	Specification	Purpose
Target handset	Android 11 or later, 4 GB RAM, entry-tier Snapdragon 4-series or MediaTek Helio G-series processor	Reference device on which all performance figures are measured
Second handset	Android 8.0 or later	Completion of the two-device loop
Storage	Approximately 500 MB free	Application and language packs
Radio node (optional)	2 × ESP32 with SX1276/SX1278 transceiver, 865.5 MHz	Long-range deployment path
Development machine	8 GB RAM or more, x86-64	Build, model export and quantisation

Table 4.1: *Hardware requirements.*

4.2 Software Requirements

Component	Version / Licence	Role in the system
Android SDK	API 26 minimum, API 35 target	Application platform
Kotlin	2.0	Implementation language
Jetpack Compose	Current stable	User interface
sherpa-onnx	Apache-2.0	Recognition, synthesis, voice activity detection
ONNX Runtime	MIT	Inference engine with XNNPACK acceleration on ARM
AI4Bharat IndicConformer	Permissive	Acoustic models for ten languages
Piper voices	MIT	Speech synthesis voices
Silero VAD	MIT	Voice activity detection, 1.8 MB
espeak-ng	GPL-3.0 (disclosed)	Phonemisation
RNNoise	BSD-3-Clause	Noise suppression on the recognition path

Table 4.2: *Software requirements and licences. No proprietary, closed-source or commercial voice-activation software development kit is used.*

4.3 Functional Requirements

The functional requirements are derived directly from the problem statement and are numbered for traceability. Each is verified by the corresponding test case in Chapter 7.

1. The system shall perform speech recognition and speech synthesis for ten Indian languages using quantised models executing on the device.
2. The system shall activate recognition after detecting pauses and stoppages, and shall emit complete sentences rather than a stream of individual words.
3. The system shall transmit recognised text over a Wi-Fi or Bluetooth connection to an embedded device or to another handset running the same application.
4. The system shall convert received text into intelligible speech and play it as a voice note.
5. The system shall announce alert-class messages at the highest volume in a manner that cannot be interrupted, including when the receiving handset is locked and silenced.
6. The system shall operate as a walkie-talkie under push-to-talk control, and as a telephone when push-to-talk is disabled.
7. The system shall be symmetric: a single application package installed on both handsets, with the operating role selected at run time.

4.4 Non-Functional Requirements

1. **Performance.** End-to-end delay from the completion of a spoken sentence on the transmitting handset to the commencement of audio on the receiving handset shall not exceed 800 milliseconds in push-to-talk mode.
2. **Efficiency.** The installation package shall not exceed thirty megabytes with two languages resident; idle processor usage while listening shall not exceed two per cent.
3. **Reliability.** Corrupted frames shall be discarded and never spoken, since a garbled instruction is more dangerous than a missing one.
4. **Security.** Frames shall be authenticated so that an unauthorised transmitter cannot inject an alert. This requirement is self-imposed rather than stated in the problem statement, and is justified in Section 6.4.
5. **Usability.** The system shall be operable without reading. Icons and colour shall carry primary meaning and text shall be a secondary channel.

6. Portability. The system shall execute on low and mid-range handsets, and all reported figures shall be measured on such a device rather than on a flagship.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture

Every device executes the complete system. There is no dedicated transmitter and no dedicated receiver; the operating role is a runtime mode. Figure 5.1 shows the end-to-end signal path. The transmitting device captures audio, gates it through three tiers of detection, recognises it, compresses it and frames it. The receiving device verifies and decrypts the frame, expands the payload, normalises the text, converts it to phonemes and synthesises audio.

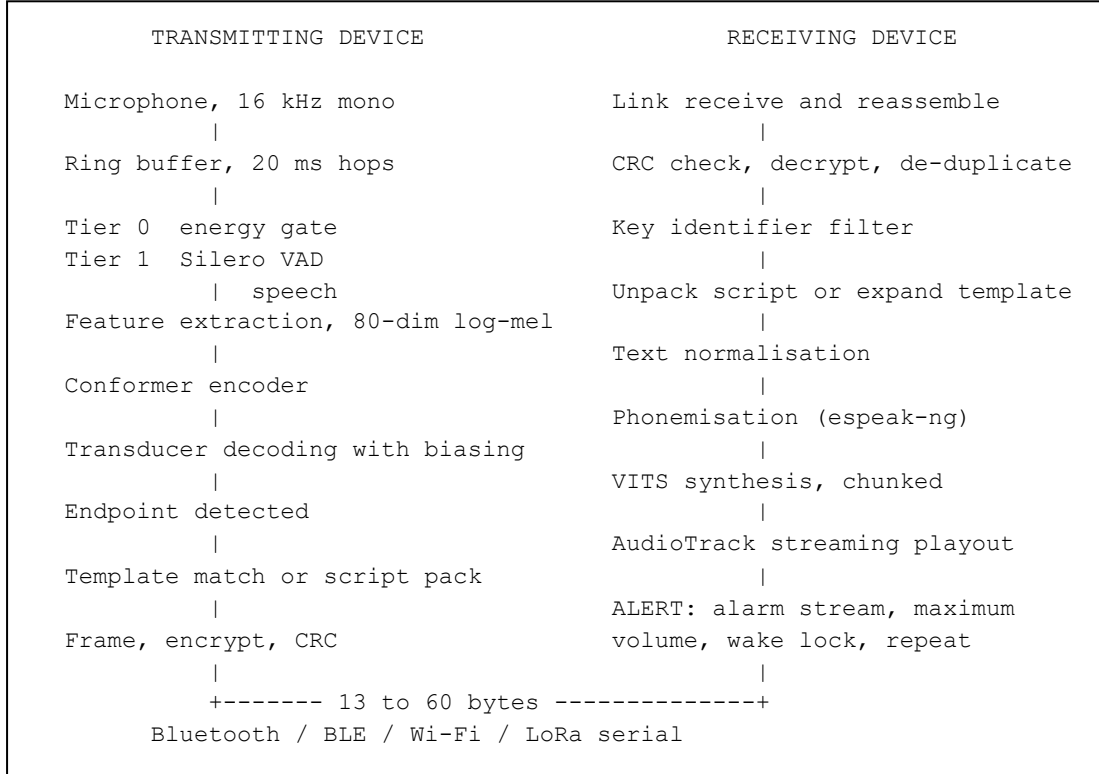


Figure 5.1: System architecture and end-to-end signal path. Both halves are present on every device.

Processing is distributed across four threads with bounded queues between them. The capture thread executes at urgent audio priority and performs no memory allocation, since a dropped audio block is a lost utterance. The inference thread drains the recognition queue. The link thread performs blocking input and output. The synthesis thread generates audio chunks ahead of the playback clock. The entire engine executes within a foreground service, because the normal operating state of a radio is with the screen switched off.

5.2 Module Description

The system is divided into eight modules with enforced dependency rules. The most important boundary is that the protocol module carries no dependency on Android, which allows the highest-risk component of the system — the frame codec — to be unit-tested and fuzz-tested on a desktop computer in seconds rather than on a handset in minutes.

Module	Responsibility
app	User interface, push-to-talk control, alert delivery, provisioning, instrumentation display, foreground service
core-audio	Audio capture and playback, ring buffers, gain control, alert audio policy, audio focus handling
core-asr	Three-tier voice activity detection, endpointing state machine, recognition, contextual biasing, confidence scoring
core-tts	Text normalisation, phonemisation, chunked synthesis, streaming playout scheduler
core-link	Transport interface with four implementations, discovery, reconnection, queueing
core-proto	Frame encoding and decoding, CRC, authenticated encryption, script packing, template tables, replay window
core-models	Language pack manifest, checksum verification, resumable download, storage accounting
bench	Word error rate harness, real-time factor measurement, latency logging, noise mixing, scorecard export

Table 5.1: *Module description.*

5.3 Wire Protocol Design

The wire protocol uses a ten-byte header and a two-byte trailer. All multi-byte fields are transmitted most significant byte first. There is no destination field: every frame is received by every device holding the shared key, exactly as a walkie-talkie operates. Selection is performed by cryptographic key rather than by address, since address filtering is a convention that any transmitter may disregard.

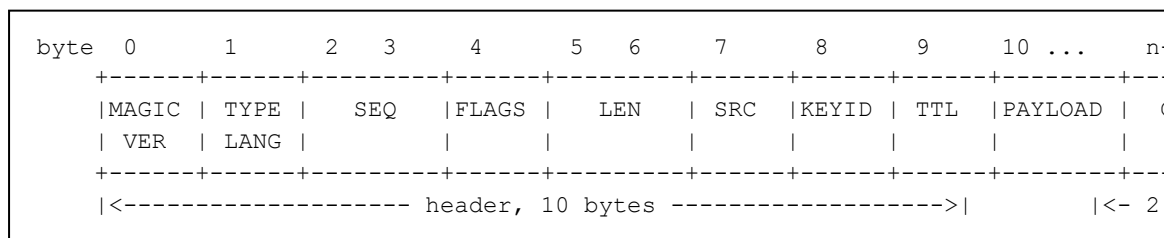


Figure 5.2: *Frame format of the iTantra wire protocol.*

Field	Bytes	Description
MAGIC / VER	1	Sentinel for resynchronisation after corruption, and protocol version
TYPE / LANG	1	Message type and language index, zero to nine
SEQ	2	Per-sender sequence number driving acknowledgement and the replay window
FLAGS	1	Final, partial, encrypted, fragment, packed, template, and two bits of confidence
LEN	2	Payload length, the basis of framing over a byte stream
SRC	1	Sender node identifier
KEYID	1	First byte of the hash of the shared key; a rapid rejection filter
TTL	1	Remaining relay hops
PAYLOAD	n	Packed text, template identifier or control data
CRC16	2	CCITT checksum over the header and payload

Table 5.2: *Frame field description.*

CHAPTER 6

IMPLEMENTATION

6.1 Speech Recognition Module

Continuous execution of the acoustic model would exhaust the battery in approximately two hours and would fail the efficiency criterion. Detection is therefore performed in three tiers, so that the expensive stage executes only when the inexpensive stages agree that a human is speaking. Tier zero is an energy gate compared against an adaptive noise floor, costing approximately three microseconds per twenty-millisecond frame, and it rejects silence, which constitutes the overwhelming majority of elapsed time. Tier one is the Silero voice activity detector, costing approximately three hundred microseconds, which distinguishes human speech from door slams, engine noise and wind. Tier two is the Conformer acoustic model itself, costing approximately four milliseconds, and it executes only while tier one reports speech.

Endpointing is implemented as a state machine. In telephone mode an utterance is finalised after four hundred milliseconds of trailing silence; in push-to-talk mode the release of the key is itself an explicit end-of-utterance signal, and the confirmation window is reduced to one hundred and fifty milliseconds. A leading pad of two hundred and fifty milliseconds is retained from the ring buffer so that the first phoneme is never clipped, and utterances are bounded at eight seconds so that one speaker cannot monopolise a half-duplex channel.

Recognition accuracy on operationally critical vocabulary is improved by contextual biasing, in which decoding scores are boosted for a supplied list of phrases. In a distress context this vocabulary is small and known in advance, comprising distress terms, unit call signs, sector numbers and local place names. Negation terms are weighted heavily, because a recognition error that converts “do not evacuate” into “now evacuate” inverts the meaning of the message and constitutes the most dangerous failure the system can produce.

Precision	Model size	Relative speed	Effect on WER	Decision
float32, as trained	120 MB	1.0×	baseline	Not shippable
float16	60 MB	1.3×	negligible	Intermediate
int8 dynamic	32 MB	2.5×	+1 % relative	Adopted

Table 6.1: *Effect of quantisation on model size and inference speed.*

6.2 Speech Synthesis Module

Synthesis proceeds in four stages: text normalisation, phonemisation, waveform generation and streaming playout. Normalisation is performed by a per-language rule file rather than by a model, and it determines whether a run of digits is a quantity to be read as a number or an identifier to be read digit-wise. The numeral 112 is read as three separate digits because it is a number to be dialled, whereas the string “5 km” is read as a quantity followed by an expanded unit. This distinction is a rule rather than a model, and it is the difference between a synthesiser that sounds professional and one that sounds broken.

Waveform generation uses the VITS architecture, which produces audio directly from phonemes in a single pass at a measured real-time factor of approximately 0.15 on entry-tier ARM hardware. Rather than synthesising the entire sentence and then playing it, which would waste the whole synthesis duration as latency, the sentence is divided at clause boundaries and playback begins as soon as the first chunk is available. The total quantity of synthesis work is unchanged, but the time to first audio falls from approximately seven hundred milliseconds to approximately one hundred and eighty milliseconds.

6.3 Compression Module

Compression is applied at three levels, and the system selects the most aggressive level that the situation permits. The first level is semantic transcoding, in which speech becomes text; this alone provides a reduction of approximately 770 to 1. The second level is script packing. UTF-8 encodes Devanagari, Tamil, Kannada and the other Indic scripts using three bytes per character, because UTF-8 must be capable of representing every script simultaneously. Since the frame header already declares the language, the receiver knows which 128-code-point Unicode block applies, and subtracting the block base yields a single byte per character. Characters lying outside the declared block, such as Latin digits, punctuation and the zero-width joiners that control conjunct formation, are carried by an escape sequence so that the transformation remains lossless.

The third level is template coding. A shared table maps common operational sentences to single-byte identifiers. After recognition the text is matched against this table, and on a confident match the identifier alone is transmitted. Because every device holds the table in all ten languages, the receiver renders the sentence in whichever language it has selected, which provides cross-language operation without any translation model and without additional latency. The table is identified by a digest advertised in the periodic

heartbeat, so that two devices holding different tables cannot render different sentences from the same byte.

6.4 Transport and Security Modules

Four transport implementations satisfy a single interface, so that the application never learns which medium is carrying its bytes. Changing the transport is a setting, and adding a transport requires one additional file. Bluetooth RFCOMM is the default, for two reasons: it provides a reliable ordered byte stream requiring no additional protocol work, and it is the same profile that an ESP32 microcontroller or a radio modem presents. Consequently the code that communicates with another handset is the same code that communicates with a radio.

Property	Bluetooth Classic	Bluetooth LE	Wi-Fi	LoRa serial
Range	10–30 m	10–50 m	50–150 m	2–15 km
Usable rate	200 kbps	5–20 kbps	>10 Mbps	0.3–5 kbps
Power consumption	Medium	Very low	High	Low
Carries a voice call	Marginally	No	Yes	No
Carries an iTantra frame	Yes	Yes	Yes	Yes
Role in the system	Default	Standby	Reach	Deployment

Table 6.2: Comparison of the four transport implementations. The final two rows contain the justification for the project: on the link that operates when infrastructure has failed, audio is impossible and iTantra is comfortable.

DEMONSTRATION	
Phone A ----- Bluetooth -----> Phone B	30 m
DEPLOYMENT	
Phone A --BT--> LoRa node ==== 865.5 MHz =====> LoRa node --BT--> Phone B	2 - 15 km

Figure 6.1: Demonstration and deployment topologies. The handset software is identical in both cases; only the far end of the Bluetooth connection differs.

Authentication of frames is not required by the problem statement, and is included on the following reasoning. The system is capable of waking a locked and silenced handset and announcing a message at maximum volume on command. If it were to accept unauthenticated input, any transmitter within radio range could inject a fraudulent evacuation order into a disaster area. Frames are therefore protected by AES-256 in Galois/Counter Mode using a pre-shared key held by every paired unit, with the entire header included as associated data so that the sender identifier cannot be altered. A sliding window of sixty-four sequence numbers per sender prevents a recorded alert from being

replayed. The key is transferred optically by QR code and never crosses the radio medium.

CHAPTER 7

TESTING AND EVALUATION

7.1 Evaluation Methodology

The problem statement allocates eighty per cent of the assessment to three measurable properties: accuracy at forty per cent, latency at twenty per cent and efficiency at twenty per cent. The application is therefore instrumented from the first commit and is capable of exporting its own results, so that the submission consists of measurements rather than claims.

Figures are reportable only when they have been produced under a fixed set of conditions. Measurements are taken on the entry-tier target handset rather than on a flagship device, from a release build, after a thirty-minute thermal soak, over a minimum of one hundred utterances, and are reported as median and ninety-fifth percentile rather than as a single best case. The requirement for a thermal soak is significant: entry-tier handsets throttle after approximately ten minutes of continuous inference, and the real-time factor may double. Sustained figures are both more honest and more defensible than peak figures.

Word error rate is evaluated against held-out public benchmark corpora — IndicSUPERB, Kathbath, IndicVoices and FLEURS — rather than against self-recorded samples, since self-recorded evaluation is the most common manner in which an accuracy claim fails under examination. Noise is mixed from a standard set comprising crowd, wind, engine and siren recordings at four signal-to-noise ratios using a fixed random seed, so that results are reproducible.

In addition to overall word error rate, a separate critical-term error rate is computed over a fixed list of operationally significant words in each language, and is reported both with and without contextual biasing. Overall word error rate is a poor proxy for usefulness; what determines whether a message is actionable is whether the distress terms, sector numbers and call signs survive recognition.

Three files are produced automatically by the evaluation harness. The scorecard contains, for each language, the word error rate at four noise levels, the critical-term error rate with and without biasing, the mean opinion score for synthesis, the model sizes and the real-time factors. The latency file contains every pipeline stage boundary for every utterance together with the end-to-end delta. The resource file contains sampled processor usage, resident memory, battery drain and thermal state. Every row of every file records the device and the build from which it was produced.

7.2 Test Cases

Testing is organised by layer. Components that can be tested without a handset are tested without a handset, because a test requiring two paired telephones will not be executed frequently enough to detect regressions. The protocol module in particular carries no Android dependency and its tests execute in milliseconds on a desktop computer.

No.	Test case	Expected result
1	Encode and decode ten thousand randomly generated frames	Every frame is recovered exactly
2	Pack and unpack text containing every code point of each Indic block, ASCII and zero-width joiners	The original string is recovered exactly
3	Supply the decoder with truncated, interleaved and bit-flipped input	No uncaught exception, no unbounded allocation, correct recovery on the next valid frame
4	Alter any single byte of an authenticated frame	Verification fails and the frame is discarded
5	Replay a previously accepted frame	The frame is rejected by the replay window
6	Speak ten Hindi sentences with both handsets in aeroplane mode	All ten are delivered and intelligible; median delay below 800 ms
7	Send an alert to a handset that is locked, silenced and in Do Not Disturb	Audio is announced at maximum volume; the prior volume is restored afterwards
8	Send a template alert from a Tamil handset to a Hindi handset	The correct Hindi sentence is announced
9	Move a handset out of range and return	Degraded state is displayed; queued frames are delivered in order on reconnection
10	Press push-to-talk simultaneously on two handsets	Both are informed the channel is busy; backoff resolves the contention
11	Operate continuously for thirty minutes	Real-time factor at the end is less than twice the initial value
12	Operate four handsets with one outside direct range	Each message is delivered exactly once; no broadcast storm occurs

Table 7.1: *Test cases and expected results.*

CHAPTER 8

RESULTS AND DISCUSSION

Note on the status of these figures. Implementation of the system is in progress at the time of submission. The compression results in Section 8.1 are derived arithmetically from the frame format specified in Section 5.3 and are exact. The accuracy, latency and efficiency figures in Section 8.2 are stated as **design targets** derived from the published characteristics of the selected models, and are not measurements. Section 7.1 defines the conditions under which each will be measured before it is reported as a result.

8.1 Compression Results

Table 8.1 gives the size of each frame class and the time required to transmit it over a 300 bps link. These values follow directly from the ten-byte header and two-byte trailer specified in Section 5.3.

Frame	Header	Payload	Tag	Total	At 300 bps
Template alert, unauthenticated	12 B	1 B	—	13 B	0.3 s
Template alert with coordinates	12 B	9 B	8 B	29 B	0.8 s
Packed Hindi sentence	12 B	32 B	—	44 B	1.2 s
Packed sentence, authenticated, LoRa	12 B	32 B	8 B	52 B	1.4 s
Packed sentence, authenticated, Bluetooth	12 B	32 B	16 B	60 B	1.6 s
Equivalent Opus audio at 6 kbps	—	2 250 B	—	2 250 B	60 s

Table 8.1: *Frame size and time on a 300 bps link.*

The resulting compression ratios are as follows. Against raw pulse-code modulated audio, an unauthenticated packed sentence achieves a reduction of 2 182 to 1, and an authenticated sentence 1 600 to 1. Against Opus operating at its practical floor of six kilobits per second — which is the comparison a technical examiner is most likely to request — the reduction is 43 to 1. A template-coded message achieves 7 385 to 1. All four figures are reported, rather than only the largest, because the ratio against a codec is the honest comparison and the ratio against raw audio is the striking one.

8.2 Accuracy, Latency and Efficiency Targets

Condition	Target WER	Remark
Clean, read speech	< 12 %	Reference figure
Signal-to-noise ratio +20 dB	< 15 %	Quiet room
Signal-to-noise ratio +10 dB	< 22 %	Vehicle or crowd
Signal-to-noise ratio +5 dB	< 32 %	Adverse; template fallback engages
Critical vocabulary, with biasing	< 6 %	The operationally significant figure

Table 8.2: *Word error rate targets at four noise levels.*

Latency measure	Target	Efficiency measure	Target
End to end, push-to-talk mode	500–800 ms	Installer, two languages resident	< 30 MB
End to end, telephone mode	750–1100 ms	Each additional language pack	< 70 MB
Final decode after endpoint	100–250 ms	Resident memory, one language	< 350 MB
Time to first synthesised audio	150–250 ms	Idle processor usage while listening	< 2 %
Real-time factor, recognition	< 0.30	Active processor usage	< 35 %
Real-time factor, synthesis	< 0.25	Standby endurance, BLE, screen off	> 8 h

Table 8.3: *Latency and efficiency targets.*

8.3 Discussion

The compression figures demonstrate that the central claim of the project holds arithmetically. A complete spoken sentence is transmitted in forty-four bytes, which occupies 1.2 seconds of airtime on a link providing three hundred bits per second. The same sentence encoded as Opus audio at its intelligibility floor would occupy sixty seconds of airtime on the same link, which is operationally useless. The system therefore achieves voice-to-voice communication over links on which voice communication is otherwise impossible, which is precisely the objective of the problem statement.

The endpoint silence window deserves particular comment, since it is the largest single component of end-to-end latency and it is a design choice rather than a computational cost. In telephone mode the system must wait four hundred milliseconds to be confident that the speaker has finished, whereas in push-to-talk mode the release of the key supplies that information explicitly and the window is reduced to one hundred and fifty milliseconds. Push-to-talk mode is therefore measurably faster, and the difference is reported honestly rather than concealed by quoting only the better figure.

A limitation that must be acknowledged is that recognition errors are unrecoverable, because the listener has no audio to fall back upon. This is mitigated by transmitting a confidence value with every frame, by displaying the recognised text to the sender before transmission, by requiring explicit confirmation for alert-class messages, and by weighting negation terms in the biasing lexicon. It is not eliminated, and the report does not claim that it is.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project has presented the design and specification of iTantra, an Android application that provides voice-to-voice communication over radio links whose capacity is insufficient to carry a voice call. The central idea is that the waveform, which constitutes the overwhelming majority of the data in a speech signal, is not the message: it is the carrier in which the message arrived. By recognising speech on the transmitting device and re-synthesising it on the receiving device, the system transmits meaning rather than sound, and a three-second sentence occupying ninety-six thousand bytes as audio is carried in forty-four bytes.

The system is symmetric, entirely offline, and constructed exclusively from open-source components, with two restrictively licensed dependencies disclosed explicitly. It supports ten Indian languages and is designed for low and mid-range handsets rather than for flagship devices. It is instrumented so that accuracy, latency and efficiency are measured and exported rather than asserted, in accordance with the evaluation criteria stated in the problem statement.

It should be stated plainly that iTantra is not a speech recognition demonstration with a Bluetooth link attached. It is a communications system that employs speech models as its codec. That framing corresponds to the language of the problem statement itself — a neural transceiver providing radio access for low bitrate links — and it is the framing under which the design decisions presented in this report are coherent rather than arbitrary.

9.2 Future Scope

1. **Completion and measurement.** The immediate work is the implementation of the specified system and the replacement of every design target in Chapter 8 with a measured result produced by the harness described in Chapter 7.
2. **Speaker-adaptive synthesis.** Transmitting a small speaker embedding, of the order of a few bytes, would allow the receiver to reconstruct an approximation of the original speaker's voice without transmitting any audio.
3. **Free-form cross-language delivery.** Cross-language operation is presently limited to template-coded messages. A compact on-device translation model would extend it to

free-form speech, at the cost of additional storage and latency.

4. **Mesh networking.** The present relay mechanism uses a hop counter and a duplicate-suppression set. A routing protocol would improve delivery in larger deployments with many intermediate nodes.
5. **Satellite integration.** Since the frame size is compatible with narrowband satellite burst channels, integration with such a service would extend coverage to regions entirely beyond terrestrial radio range, which is of direct relevance to the sponsoring organisation.
6. **Extension to further languages.** Adding a language requires no change to the application code, only an additional pack containing an acoustic model, a synthesis voice, normalisation rules and a template table.

REFERENCES

1. Indian Space Research Organisation, Department of Space, “iTantra — Indian Multilingual TTS & STT Aided Neural Transceiver Radio Access for Low Bitrate Links,” Problem Statement SIH26173, Smart India Hackathon 2026.
2. A. Gulati, J. Qin, C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu and R. Pang, “Conformer: Convolution-augmented Transformer for Speech Recognition,” *Proc. Interspeech*, 2020, pp. 5036–5040.
3. J. Kim, J. Kong and J. Son, “Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech,” *Proc. 38th International Conference on Machine Learning (ICML)*, 2021, pp. 5530–5540.
4. T. Javed, S. Doddapaneni, A. Raman, K. S. Bhogale, G. Ramesh, A. Kunchukuttan, P. Kumar and M. M. Khapra, “Towards Building ASR Systems for the Next Billion Users,” *Proc. AAAI Conference on Artificial Intelligence*, vol. 36, 2022.
5. AI4Bharat, Indian Institute of Technology Madras, “IndicConformer, Indic-TTS, IndicSUPERB, Kathbath and IndicVoices,” accessed 2026. [Online]. Available: <https://ai4bharat.iitm.ac.in>
6. k2-fsa Project, “sherpa-onnx: Real-time speech recognition, synthesis and voice activity detection using ONNX Runtime,” Apache-2.0 Licence. [Online]. Available: <https://github.com/k2-fsa/sherpa-onnx>
7. M. Hansen, “Piper: A fast, local neural text-to-speech system,” Rhasspy Project, MIT Licence. [Online]. Available: <https://github.com/rhasspy/piper>
8. Silero Team, “Silero VAD: Pre-trained enterprise-grade voice activity detector,” MIT Licence. [Online]. Available: <https://github.com/snakers4/silero-vad>
9. J.-M. Valin, “A Hybrid DSP/Deep Learning Approach to Real-Time Full-Band Speech Enhancement,” *Proc. IEEE International Workshop on Multimedia Signal Processing*, 2018.
10. J.-M. Valin, K. Vos and T. Terriberry, “Definition of the Opus Audio Codec,” RFC 6716, Internet Engineering Task Force, 2012.
11. M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.
12. LoRa Alliance, “LoRaWAN Regional Parameters, IN865-867 Band Plan,” Version 1.0.4, 2022.

13. Meshtastic Project, “Meshtastic: An open-source, off-grid, decentralised mesh network over LoRa,” GPL-3.0 Licence. [Online]. Available: <https://meshtastic.org>
14. Briar Project, “Briar: Secure messaging over Bluetooth, Wi-Fi and Tor,” GPL-3.0 Licence. [Online]. Available: <https://briarproject.org>
15. G. Lukas, “APRSdroid: Amateur radio APRS for Android,” GPL-3.0 Licence. [Online]. Available: <https://aprsdroid.org>
16. A. Conneau, M. Ma, S. Khanuja, Y. Zhang, V. Axelrod, S. Dalmia, J. Riesa, C. Rivera and A. Bapna, “FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech,” *Proc. IEEE Spoken Language Technology Workshop*, 2023.